

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

Отчет о программном проекте

на тему: _____ Нейронные сети с нуля _____

Выполнил:

Студент группы БПМИ245 _____

_____ подпись студента _____

_____ Аветисов Тимофей Михайлович _____

_____ Подпись _____

_____ И.О.Фамилия _____

_____ 24.05.2026 _____

_____ Дата _____

Принял:

Руководитель проекта _____

_____ Трушин Дмитрий Витальевич, к.ф.-м.н. _____

_____ Имя, Отчество, Фамилия _____

_____ Доцент _____

_____ Должность, ученое звание _____

_____ ФКН НИУ ВШЭ _____

_____ Место работы (Компания или подразделение НИУ ВШЭ) _____

Дата проверки _____ 2026 _____

_____ Оценка (по 10-ти бальной шкале) _____

_____ подпись руководителя _____

_____ Подпись _____

Москва 2026

Содержание

1	Введение	3
2	Функциональные и нефункциональные требования	4
2.1	Назначение библиотеки	4
2.2	Функциональные требования	4
2.3	Нефункциональные требования	5
3	Математическая постановка	5
3.1	Общий принцип обучения и устройства нейронной сети	5
3.2	Представление данных	5
3.3	Линейный слой	6
3.4	Функции активации	6
3.5	Функции потерь	6
3.6	Softmax и кросс-энтропия	7
3.7	Обратное распространение ошибки	8
4	Общая архитектура проекта	8
4.1	Структура директорий	8
4.2	Типы данных	9
4.3	Внешние зависимости	10
5	Классы модели и обобщенные интерфейсы	10
5.1	Основные классы библиотеки	10
5.2	Классы слоев и сети	10
5.3	Стирание типов: контракты компонентов	11
5.4	Поток данных при обучении	12
5.5	Логика владения кэшами	13
6	Базовые типы и генерация случайных величин	14
6.1	Базовые числовые типы	14
6.2	RandomGenerator	14
7	Функции потерь	14
7.1	Общий контракт	14
7.2	Loss-функции для регрессии	14
7.3	Loss-функции для классификации	14
8	Оптимизаторы и scheduler-ы	15
8.1	Learning rate scheduler	15
8.2	SGDOptimizer	15
8.3	AdamOptimizer	15
9	Работа с данными	16
9.1	DataLoader	16
9.2	CSV-загрузка	16
10	Trainer и цикл обучения	16
10.1	Роль Trainer	16
10.2	Алгоритм одной эпохи	16
10.3	Состояние оптимизатора	17
11	Контракты API и обработка ошибок	17
11.1	Стратегия обработки ошибок	17

12	Размерности, сложность и численная устойчивость	17
12.1	Соглашения о размерностях	17
12.2	Асимптотическая сложность	18
12.3	Память и кэши	18
12.4	Численная устойчивость	18
13	Пример пользовательского сценария	19
13.1	Минимальный пример обучения	19
14	Экспериментальное обучение на MNIST	19
14.1	Конфигурация эксперимента	19
14.2	Методика расчета метрик	21
14.3	Результаты обучения	21
15	Сборка проекта	22
15.1	Сборка через CMake	22
15.2	Сборка через just	22
16	Тестирование	23
16.1	Организация тестового набора	23
16.2	Проверяемые сценарии	23
16.3	Запуск тестов	23
16.4	Статические соглашения о стиле	23
17	Текущие ограничения	24
17.1	Ограничения модели	24
17.2	Ограничения производительности	24
18	Заключение	24

Аннотация

Проект посвящен разработке библиотеки NNS для построения и обучения полносвязных нейронных сетей на языке C++23 без использования готовых фреймворков машинного обучения. В отчете рассматриваются математическая модель прямого и обратного прохода, архитектура программной реализации, организация обобщенных интерфейсов через стирание типов, реализация функций потерь, оптимизаторов, загрузки данных и обучающего цикла. Приводится пример обучения на датасете MNIST.

1 Введение

Нейронные сети являются одним из основных инструментов современного машинного обучения. Их применяют в задачах классификации, регрессии, распознавания изображений, обработки сигналов, анализа естественного языка и в других областях, где зависимость между входными признаками и целевой переменной имеет сложный нелинейный характер. Практическое распространение нейронных сетей связано не только с развитием вычислительных ресурсов, но и с появлением удобных библиотек, скрывающих детали линейной алгебры, автоматического дифференцирования, оптимизации и обработки данных.

Высокоуровневые фреймворки позволяют быстро строить модели, однако при их использовании часть существенных технических деталей остается скрытой за готовыми абстракциями. В частности, пользователь часто не работает напрямую с формами матриц, порядком вычисления градиентов, устройством кэшей прямого прохода, состоянием оптимизатора и численной устойчивостью функций потерь. В данном проекте эти механизмы реализованы явно: зафиксированы соглашения о представлении данных, выведены формулы обратного распространения ошибки и спроектированы интерфейсы, которые допускают расширение системы новыми компонентами.

Цель проекта состоит в разработке header-only библиотеки NNS, предназначенной для построения и обучения полносвязных нейронных сетей на C++23. Под выражением «с нуля» в рамках работы понимается отказ от специализированных библиотек машинного обучения: проект не использует PyTorch, TensorFlow, Keras и аналогичные средства. При этом используются библиотеки общего назначения: Eigen для матричных операций, проху для стирания типов и Catch2 для автоматического тестирования.

В результате работы реализована header-only библиотека, в которой пользователь может задать последовательность линейных слоев и скалярных функций активации, выбрать функцию потерь, оптимизатор и стратегию изменения шага обучения, загрузить данные из матриц или CSV-файла, а затем запустить обучение. Архитектура построена вокруг строгого разделения ролей: слой вычисляет локальное преобразование и локальный градиент, нейронная сеть организует прямой и обратный проход по слоям, оптимизатор обновляет параметры, а тренер связывает модель, данные, функцию потерь и оптимизатор в единый цикл обучения.

В отчете сначала формулируются требования к программному проекту, затем вводится математическая модель полносвязной сети и ее обучения. Далее описывается программная архитектура библиотеки, внутренние соглашения о представлении данных, устройство слоев, функций потерь, оптимизаторов, загрузчика данных и тренера. После этого рассматриваются пример использования на задаче классификации изображений MNIST, организация сборки и тестирования.

2 Функциональные и нефункциональные требования

2.1 Назначение библиотеки

Библиотека NNS предназначена для создания, обучения и тестового применения полносвязных нейронных сетей. Основным пользовательским сценарием является следующий процесс: пользователь формирует обучающую выборку в виде матриц признаков и целевых переменных, создает объект нейронной сети из набора слоев, выбирает функцию потерь и оптимизатор, передает эти объекты в тренер и получает историю значения функции потерь по эпохам.

Система проектируется как библиотека, а не как отдельное приложение. Это означает, что основная ценность проекта заключается в предоставляемом API: классы и функции должны быть пригодны для подключения к стороннему CMake-проекту, для написания собственных экспериментов и для расширения пользовательскими компонентами. Файл `main.cpp` в репозитории является примером применения библиотеки, а не обязательной частью ее публичного интерфейса.

2.2 Функциональные требования

К функциональным требованиям проекта относятся:

1. поддержка базовых числовых типов и матричного представления данных;
2. создание линейных слоев с настраиваемыми размерами входа и выхода;
3. инициализация весов различными распределениями;
4. поддержка встроенных функций активации ReLU, Sigmoid и Tanh;
5. возможность добавления пользовательских скалярных функций активации;
6. возможность выполнять предсказание, прямой проход и обратный проход в процессе обучения;
7. поддержка нескольких функций потерь для задач регрессии и классификации;
8. поддержка оптимизаторов SGD и Adam;
9. поддержка постоянного и убывающего шага обучения;
10. загрузка данных из CSV-файла с выделением целевых колонок;
11. разбиение данных на батчи с опциональным перемешиванием;
12. запуск обучения на заданное число эпох;
13. возврат истории среднего значения функции потерь по эпохам;
14. наличие автоматических тестов для основных компонентов.

Отдельным требованием является расширяемость. Пользователь должен иметь возможность определить собственную функцию потерь, оптимизатор, scheduler шага обучения или функцию активации при условии, что пользовательский тип реализует ожидаемый набор методов. Для этого в проекте используются type-erased обертки на основе библиотеки `proxy`.

2.3 Нефункциональные требования

К нефункциональным требованиям относятся переносимость, воспроизводимость, простота сборки и явная диагностика ошибок. Проект написан на C++23 и собирается через CMake. Библиотека реализована как INTERFACE-цель, поэтому ее заголовки могут подключаться без отдельного этапа компиляции статической или динамической библиотеки.

Воспроизводимость обеспечивается классом `RandomGenerator`, который хранит генератор псевдослучайных чисел и позволяет явно задавать seed. Это важно как для инициализации весов, так и для перемешивания данных в `DataLoader`. При одинаковом seed и одинаковом порядке вызовов пользователь должен получать одинаковую последовательность случайных величин.

Диагностика ошибок реализуется через исключения стандартной библиотеки. Компоненты проверяют размеры матриц, непустоту входов, корректность гиперпараметров и согласованность кэшей. Такой подход повышает надежность библиотеки: ошибка формы данных обнаруживается в точке нарушения контракта, а не приводит к неявно неверному результату в конце обучения.

3 Математическая постановка

3.1 Общий принцип обучения и устройства нейронной сети

Рассматривается задача обучения с учителем. Дана выборка

$$\mathcal{D} = \{(x^{(j)}, y^{(j)})\}_{j=1}^N,$$

где $x^{(j)} \in \mathbb{R}^d$ — вектор признаков, а $y^{(j)}$ — целевая переменная. Требуется построить параметрическое отображение

$$f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^k,$$

которое минимизирует среднюю ошибку на обучающей выборке.

Полносвязная нейронная сеть задается как композиция преобразований

$$f_\theta = f_m \circ f_{m-1} \circ \dots \circ f_1.$$

Каждое преобразование называется слоем. В полносвязной сети параметризованные слои являются аффинными преобразованиями, а нелинейность вносится функциями активации. Если обозначить $h_0 = x$, то прямой проход имеет вид

$$h_i = f_i(h_{i-1}), \quad i = 1, \dots, m.$$

Выход h_m является предсказанием модели.

Обучение состоит в минимизации эмпирического риска

$$J(\theta) = \frac{1}{N} \sum_{j=1}^N \ell(f_\theta(x^{(j)}), y^{(j)}),$$

где ℓ — функция потерь. На каждой итерации сначала вычисляется предсказание, затем значение функции потерь, после чего по правилу дифференцирования композиции находятся градиенты по параметрам. Далее параметры изменяются в направлении уменьшения эмпирического риска.

3.2 Представление данных

Батч — это поднабор объектов $x^{(i_1)}, x^{(i_2)}, \dots, x^{(i_B)}$ и соответствующих им целевых переменных $y^{(i_1)}, y^{(i_2)}, \dots, y^{(i_B)}$. Для батча из B объектов удобно объединить входы в матрицу

$$X = [x^{(1)} \quad x^{(2)} \quad \dots \quad x^{(B)}] \in \mathbb{R}^{d \times B}.$$

Каждый столбец соответствует одному объекту. Если выходная размерность равна k , то целевые значения батча можно записать как

$$Y = [y^{(1)} \quad y^{(2)} \quad \dots \quad y^{(B)}] \in \mathbb{R}^{k \times B}.$$

В задачах многоклассовой классификации $y^{(j)}$ часто является one-hot вектором класса: вектором длины, равной числу классов, с нулями во всех позициях, кроме позиции истинного класса. Также целевое значение может задаваться меткой класса. В задачах регрессии $y^{(j)}$ содержит одно или несколько вещественных целевых значений.

3.3 Линейный слой

Пусть вход батча имеет размерность $X \in \mathbb{R}^{d_{in} \times B}$. Линейный слой с параметрами

$$A \in \mathbb{R}^{d_{out} \times d_{in}}, \quad b \in \mathbb{R}^{d_{out}}$$

задает преобразование

$$Z = AX + b\mathbf{1}^T,$$

где $\mathbf{1} \in \mathbb{R}^B$ — вектор из единиц. Слагаемое $b\mathbf{1}^T$ означает, что один и тот же вектор смещения прибавляется ко всем столбцам батча.

Пусть известен градиент функции потерь по выходу слоя:

$$G_Z = \frac{\partial L}{\partial Z} \in \mathbb{R}^{d_{out} \times B}.$$

Тогда градиенты по параметрам и входу выражаются формулами

$$\frac{\partial L}{\partial A} = G_Z X^T, \quad \frac{\partial L}{\partial b} = G_Z \mathbf{1}, \quad \frac{\partial L}{\partial X} = A^T G_Z.$$

Эти формулы следуют из дифференциала $dZ = dA \cdot X + A \cdot dX + db \cdot \mathbf{1}^T$.

3.4 Функции активации

Если сеть состоит только из аффинных преобразований, то вся композиция также является аффинным преобразованием. Поэтому между параметризованными слоями вводятся нелинейные функции активации. Обычно они применяются поэлементно:

$$H_{ij} = \varphi(Z_{ij}).$$

В качестве примеров используются следующие функции. ReLU задается как

$$\text{ReLU}(x) = \max(0, x),$$

а ее производная почти всюду равна

$$\text{ReLU}'(x) = \begin{cases} 1, & x > 0, \\ 0, & x \leq 0. \end{cases}$$

Сигмоида и ее производная имеют вид

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \quad \sigma'(x) = \sigma(x)(1 - \sigma(x)).$$

Гиперболический тангенс задается формулой

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}},$$

а его производная равна

$$\tanh'(x) = 1 - \tanh^2(x).$$

Эти функции подробно рассматриваются в литературе по глубокому обучению [4].

3.5 Функции потерь

Функция потерь $\ell(\hat{y}, y)$ измеряет расхождение между предсказанием $\hat{y}$ и целевым значением y .

Для регрессии часто используется среднеквадратичная ошибка:

$$L_{MSE}(\hat{Y}, Y) = \frac{1}{n} \sum_{i,j} (\hat{Y}_{ij} - Y_{ij})^2,$$

где n — число элементов матрицы. Ее градиент по предсказанию:

$$\frac{\partial L_{MSE}}{\partial \hat{Y}} = \frac{2}{n} (\hat{Y} - Y).$$

Средняя абсолютная ошибка задается формулой

$$L_{MAE}(\hat{Y}, Y) = \frac{1}{n} \sum_{i,j} |\hat{Y}_{ij} - Y_{ij}|.$$

Функция Хьюбера для ошибки $a = \hat{y} - y$ определяется как

$$L_{\delta}(a) = \begin{cases} \frac{1}{2}a^2, & |a| \leq \delta, \\ \delta(|a| - \frac{1}{2}\delta), & |a| > \delta. \end{cases}$$

Она ведет себя как квадратичная ошибка при малых отклонениях и как абсолютная ошибка при больших отклонениях.

Для бинарной классификации используется бинарная кросс-энтропия

$$L_{BCE}(p, y) = -y \log p - (1 - y) \log(1 - p),$$

где $p \in (0, 1)$ — предсказанная вероятность класса 1. Если модель возвращает $\logit z$, то $p = \sigma(z)$. В этом случае кросс-энтропию можно записать напрямую через z :

$$L(z, y) = \max(z, 0) - zy + \log(1 + e^{-|z|}).$$

Такая запись численно устойчивее прямого вычисления $-\log \sigma(z)$ или $-\log(1 - \sigma(z))$, поскольку не требует вычислять логарифм вероятности, которая из-за округления может стать слишком близкой к нулю.

3.6 Softmax и кросс-энтропия

В многоклассовой классификации модель возвращает вектор logits $z \in \mathbb{R}^k$. Чтобы получить распределение вероятностей по классам, применяется softmax:

$$p_i = \frac{e^{z_i}}{\sum_{r=1}^k e^{z_r}}, \quad i = 1, \dots, k.$$

Для one-hot вектора y кросс-энтропия равна

$$L(z, y) = - \sum_{i=1}^k y_i \log p_i.$$

Если истинный класс имеет номер c , то это выражение сводится к

$$L(z, y) = - \log p_c.$$

На практике softmax и кросс-энтропию целесообразно рассматривать как единую функцию потерь, принимающую logits. Причина состоит в численной устойчивости. Экспоненты e^{z_i} могут переполниться при больших положительных z_i . Поэтому используется тождественное преобразование

$$\text{softmax}(z) = \text{softmax}(z - \alpha \mathbf{1})$$

для любой константы α . Обычно выбирают

$$\alpha = \max_i z_i.$$

Тогда все аргументы экспоненты неположительны, а максимальный равен нулю:

$$p_i = \frac{e^{z_i - \alpha}}{\sum_{r=1}^k e^{z_r - \alpha}}.$$

Кроме того, производная объединенной функции имеет простой вид. Для одного объекта

$$\frac{\partial L}{\partial z_i} = p_i - y_i.$$

Для батча размера B при усреднении по объектам градиент делится на B . Поэтому хранить softmax как отдельный слой перед кросс-энтропией не обязательно: математически корректнее и численно устойчивее объединить эти две операции в одну функцию потерь, работающую с logits.

3.7 Обратное распространение ошибки

Обратное распространение ошибки является применением правила дифференцирования композиции функций к последовательности слоев. Подробный вывод алгоритма backpropagation для полносвязных сетей приведен, например, в книге М. Nielsen [10].

Пусть

$$h_i = f_i(h_{i-1}; \theta_i), \quad i = 1, \dots, m,$$

а итоговая функция потерь равна

$$L = \ell(h_m, y).$$

Обозначим

$$G_i = \frac{\partial L}{\partial h_i}.$$

Тогда обратный проход вычисляет градиенты в порядке $i = m, m-1, \dots, 1$:

$$G_{i-1} = \left(\frac{\partial h_i}{\partial h_{i-1}} \right)^T G_i.$$

Если слой имеет параметры θ_i , то одновременно вычисляется

$$\frac{\partial L}{\partial \theta_i} = \left(\frac{\partial h_i}{\partial \theta_i} \right)^T G_i.$$

После вычисления градиентов параметры обновляются оптимизационным методом. В простейшем случае градиентного спуска

$$\theta_{t+1} = \theta_t - \eta_t \nabla_{\theta} J(\theta_t),$$

где $\eta_t > 0$ — шаг обучения. Более сложные методы, например Adam, используют дополнительные оценки моментов градиента, но также основаны на градиенте эмпирического риска.

4 Общая архитектура проекта

4.1 Структура директорий

Публичные заголовки библиотеки расположены в директории `include/nns`. Тесты находятся в папке `tests`. Документация расположена в `docs`, а отчет — в `docs/formal`. Главный публичный заголовок `include/nns/NNS.hpp` включает все основные модули библиотеки и предназначен для пользовательского кода.

Компонентная архитектура NNS

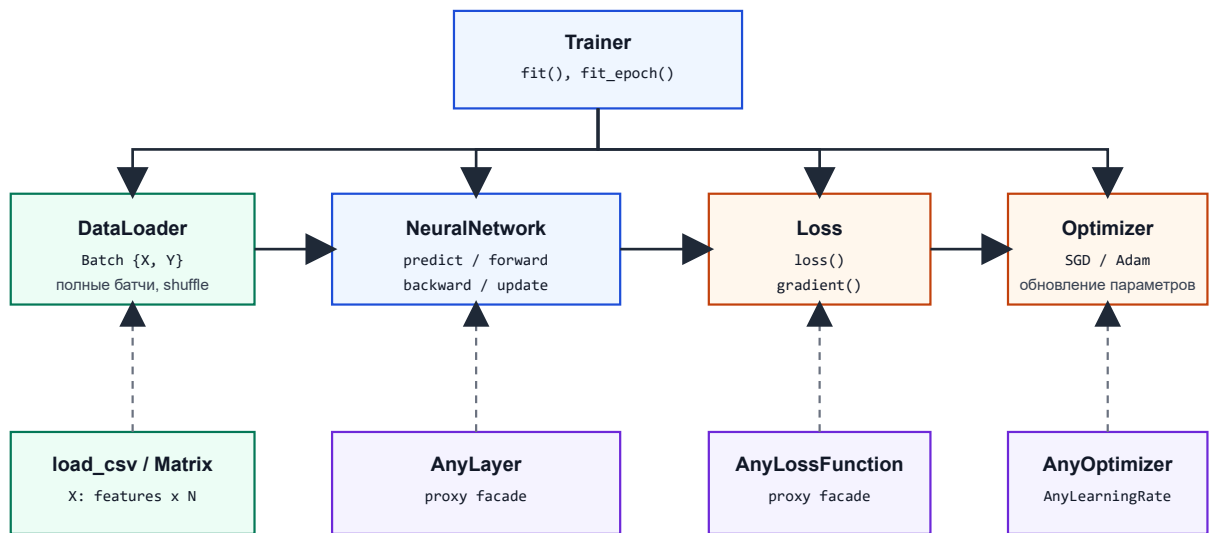


Рис. 1: Компонентная архитектура библиотеки NNS

Основные группы заголовков:

- `core` — базовые типы и `strong type`-обертки;
- `activation` — функции активации и `type-erased` интерфейс активации;
- `layers` — линейный слой, слой активации и `type-erased` интерфейс слоя;
- `network` — класс нейронной сети;
- `loss` — функции потерь и их обобщенный интерфейс;
- `learningrates` — стратегии изменения шага обучения;
- `optimizer` — оптимизаторы и их обобщенный интерфейс;
- `trainer` — цикл обучения;
- `utils` — генератор случайных чисел, загрузчик данных и CSV-парсер.

На рисунке 1 приведена компонентная схема библиотеки. Центральным объектом в процессе обучения является `Trainer`: он получает батчи от `DataLoader`, вызывает прямой и обратный проход `NeuralNetwork`, использует `Loss` для вычисления значения ошибки и начального градиента, а затем передает градиенты в `Optimizer`.

4.2 Типы данных

В коде используются базовые типы, заданные в `include/nns/core/Types.hpp`:

```
1 using Scalar = double;
2 using Matrix = Eigen::Matrix<Scalar, Eigen::Dynamic, Eigen::Dynamic>;
3 using Vector = Eigen::Matrix<Scalar, Eigen::Dynamic, 1>;
4 using Index = Eigen::Index;
```

Тип `Scalar` задает числовой тип библиотеки. `Matrix` используется для батчей, весов и градиентов весов. `Vector` используется для смещений линейных слоев и их градиентов.

4.3 Внешние зависимости

Для матричных операций используется Eigen версии 3.4.0 [2]. Eigen предоставляет динамические матрицы, векторизованные операции, поэлементные выражения и удобные средства для работы с блоками матриц. Это позволяет реализовать вычисления нейронной сети без ручного управления памятью для двумерных массивов.

Для стирания типов используется библиотека `proxu` версии 4.0.0 [9]. Она позволяет описывать фасады с требуемыми методами и хранить объекты разных типов за единым интерфейсом без классического наследования с виртуальными методами. В проекте этот механизм применяется для активаций, слоев, функций потерь, оптимизаторов и `scheduler`-ов.

Для тестирования используется Catch2 версии 3.7.1 [1]. Тесты собираются как отдельный исполняемый файл `nns_tests` и регистрируются в CTest через `catch_discover_tests`. Зависимости подтягиваются через CMake `FetchContent` [7].

5 Классы модели и обобщенные интерфейсы

5.1 Основные классы библиотеки

Публичный API библиотеки состоит из нескольких групп классов. Их роли разделены так, чтобы модель, данные, функция потерь и процесс обучения могли тестироваться и заменяться независимо.

- `RandomGenerator` — генератор случайных чисел, перемешивание индексов и инициализация матриц;
- `LinearLayer` — Линейный слой;
- `ActivationLayer` — Слой применяющий функцию активации;
- `NeuralNetwork` — Нейронная сеть;
- `MSELoss`, `MAELoss`, `HuberLoss`, `BCELoss`, `CrossEntropyLoss` — функции потерь;
- `ConstantLearningRate` и `ExponentialLearningRate` — `scheduler`-ы шага обучения;
- `SGDOptimizer` и `AdamOptimizer` — оптимизаторы параметров;
- `DataLoader` — разбиение матриц данных на батчи;
- `Trainer` — координатор цикла обучения.

Для расширяемых точек API используются обобщенные оболочки: `AnyScalarActivation`, `AnyLayer`,

`AnyLossFunction`, `AnyOptimizer` и `AnyLearningRateScheduler`. Они позволяют хранить пользовательские типы за единым интерфейсом без общего базового класса.

5.2 Классы слоев и сети

Класс `LinearLayer` хранит матрицу весов `A_` и вектор смещения `b_`. Конструктор принимает входную и выходную размерность, так же необязательные параметры генератор случайных чисел, схему распределения(по умолчанию нормальное распределение) и коэффициент масштаба(по умолчанию 1):

```
1 nns::LinearLayer layer(  
2     nns::In{784},  
3     nns::Out{128},  
4     rng,  
5     nns::Distribution::HeNormal,  
6     nns::Gain{1.0});
```

При создании слоя проверяется, что обе размерности положительны. Затем создается матрица весов размера `out x in`, а вектор смещения инициализируется нулями. Нулевая инициализация смещения не создает симметрию между нейронами, поскольку симметрия уже нарушается случайной инициализацией весов.

Класс `ActivationLayer` хранит `AnyScalarActivation`. Его задача — применить скалярную функцию активации к каждому элементу матрицы. Такой слой не имеет обучаемых параметров, но участвует в обратном проходе: входящий градиент умножается на производную функции активации в точке, сохраненной при прямом проходе.

Класс `NeuralNetwork` хранит последовательность `AnyLayer`. Конструктор является `variadic template` и принимает произвольную последовательность объектов, которые можно преобразовать в слой:

```

1 nns::NeuralNetwork net(
2     nns::LinearLayer(
3         nns::In{784},
4         nns::Out{128},
5         rng,
6         nns::Distribution::HeNormal),
7     nns::ReLU(),
8     nns::LinearLayer(
9         nns::In{128},
10        nns::Out{10},
11        rng,
12        nns::Distribution::XavierNormal));

```

Линейные слои передаются как готовые объекты, а скалярные функции активации автоматически оборачиваются в `ActivationLayer`. В результате пользовательский код остается близким к математическому описанию сети: линейное преобразование, нелинейность, следующее линейное преобразование.

5.3 Стирание типов: контракты компонентов

Нейронная сеть должна хранить слои разных типов в одном контейнере. Такая же задача возникает для функций потерь, оптимизаторов и `scheduler`-ов. В проекте для этого используется стирание типов через `proxy`. Конкретный тип объекта скрывается, но требуемый набор методов фиксируется фасадом(интерфейсом).

Для скалярной функции активации требуются два метода:

```

1 Scalar forward(Scalar x) const;
2 Scalar derivative(Scalar x) const;

```

Фасад `ScalarActivation` описывает эти операции, а тип `AnyScalarActivation` хранит объект любой функции активации, удовлетворяющей этому интерфейсу. Например, пользователь может определить собственную функцию `LeakyReLU`:

```

1 struct LeakyReLU {
2     nns::Scalar forward(nns::Scalar x) const {
3         return x > 0.0 ? x : 0.01 * x;
4     }
5
6     nns::Scalar derivative(nns::Scalar x) const {
7         return x > 0.0 ? 1.0 : 0.01;
8     }
9 };

```

Слой должен поддерживать четыре операции:

```

1 std::pair<Matrix, std::any> forward(Matrix&& X);
2 Matrix predict(const Matrix& X) const;
3 std::pair<Matrix, std::any> backward(
4     Matrix&& dY,
5     const std::any& cache);
6 std::any update(
7     std::any&& grads,
8     AnyOptimizer& opt,
9     std::any&& opt_cache);

```

Метод `predict` используется для предсказания и не создает и не сохраняет кэш. Метод `forward` используется при обучении и возвращает кэш для последующего обратного прохода. Метод `backward` принимает градиент по выходу слоя и кэш прямого прохода, а возвращает градиент по входу и градиенты параметров. Метод `update` применяет оптимизатор к параметрам слоя и возвращает новое состояние оптимизатора для этого слоя.

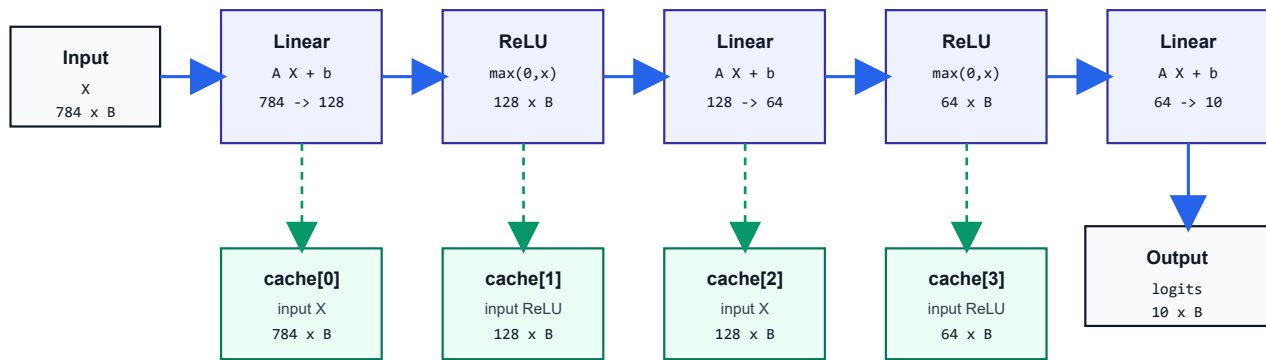
Функция потерь предоставляет два метода:

```

1 Scalar loss(const Matrix& y_hat, const Matrix& y);
2 Matrix gradient(const Matrix& y_hat, const Matrix& y);

```

Прямой проход и формирование кэшей



forward возвращает пару: выходная матрица и `std::vector<std::any>` с локальными кэшами слоев.

Рис. 2: Прямой проход по последовательности слоев

Оптимизатор обновляет матричные и векторные параметры:

```
1 std::any update_weights(  
2     Matrix& param,  
3     const Matrix& grad,  
4     std::any&& cache);  
5 std::any update_weights(  
6     Vector& param,  
7     const Vector& grad,  
8     std::any&& cache);  
9 void iter_step();
```

Scheduler шага обучения предоставляет текущий шаг, увеличивает номер итерации и возвращает номер текущей итерации:

```
1 Scalar get_lr();  
2 void iter_step();  
3 size_t get_iter() const;
```

Такое разделение не смешивает расписание шага обучения с правилом обновления параметров. Один и тот же scheduler может использоваться в разных оптимизаторах.

5.4 Поток данных при обучении

Обучение одного батча строится как последовательность трех проходов по сети. Сначала `NeuralNetwork::forward` передает входную матрицу по слоям слева направо. Каждый слой возвращает выходную матрицу и свой локальный кэш:

```
1 auto [Y, cache] = layer->forward(std::move(X));  
2 X = std::move(Y);  
3 layers_cache.push_back(std::move(cache));
```

Сеть не интерпретирует содержимое локального кэша. Она только сохраняет кэши в `std::vector<std::any>` в том же порядке, в котором расположены слои.

После вычисления значения функции потерь вызывается `Loss::gradient`. Полученный градиент является начальным градиентом для `NeuralNetwork::backward`. Обратный проход идет по слоям справа налево. Для слоя с индексом

Обратный проход и обновление параметров

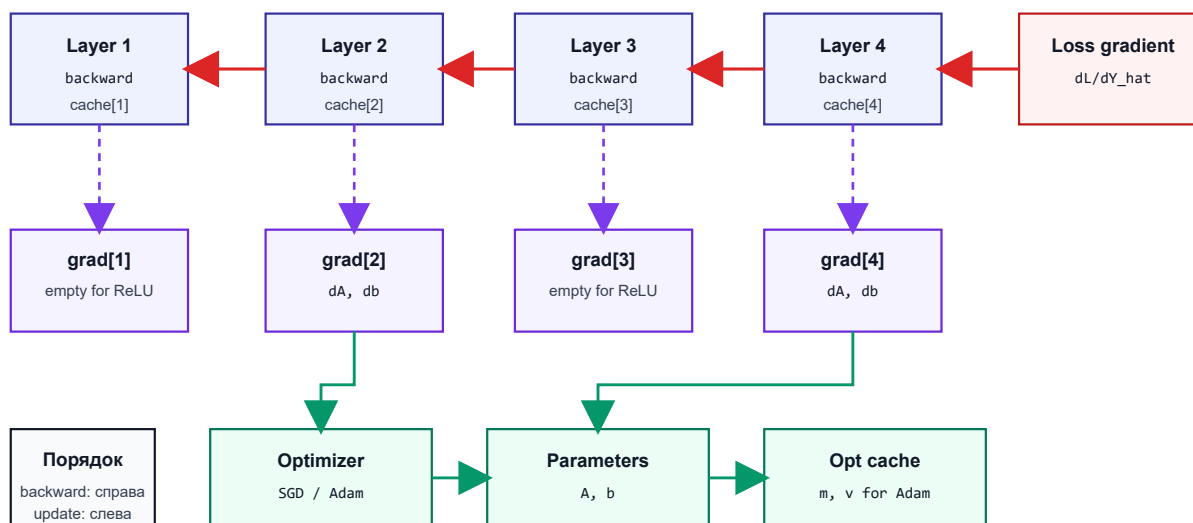


Рис. 3: Обратный проход и обновление параметров

`i` сеть берет кэш с тем же индексом, который был создан этим слоем в прямом проходе. Таким образом, слой получает ровно тот локальный объект состояния, который он сам вернул из `forward`.

Линейный слой сохраняет входную матрицу батча, потому что она нужна для вычисления градиента по весам. Слой активации также сохраняет вход, так как производная активации вычисляется в точке прямого прохода. При этом типы кэшей у разных слоев не обязаны совпадать: конкретный слой сам знает, какой тип он положил в `std::any`, и сам извлекает его в `backward`.

Результатом обратного прохода является градиент по входу сети и набор градиентов по слоям. Для линейного слоя градиент параметров представлен парой `Matrix, Vector`: градиентом по матрице весов и градиентом по вектору смещения. У слоя активации обучаемых параметров нет, поэтому его градиент параметров пустой.

После этого `NeuralNetwork::update` идет по слоям слева направо и передает каждому слою его градиенты, оптимизатор и локальное состояние оптимизатора. Состояние оптимизатора отделено от кэша прямого прохода. Кэш прямого прохода нужен только для одного шага `forward -> backward`; состояние оптимизатора живет между итерациями обучения и хранится у `Trainer`.

5.5 Логика владения кэшами

`std::any` используется для хранения объектов, тип которых известен только конкретному компоненту. Для кэша прямого прохода действует простое правило: объект, созданный слоем в `forward`, должен вернуться в тот же слой в `backward`. Сеть обеспечивает порядок передачи, но не проверяет внутренний тип кэша.

Такой подход сохраняет локальность реализации. Если добавляется новый слой, не требуется менять общий тип кэша или расширять общий `std::variant`. Новый слой определяет собственный внутренний тип кэша, возвращает его из `forward` и извлекает его в `backward`.

Разделение кэшей также фиксирует границы ответственности. Слой отвечает за локальную математику и локальный кэш. `NeuralNetwork` отвечает за порядок обхода слоев. `Trainer` связывает сеть, данные, функцию потерь и оптимизатор, но не интерпретирует содержимое кэшей слоев.

6 Базовые типы и генерация случайных величин

6.1 Базовые числовые типы

Все вычисления в текущей версии выполняются в типе `double`. Этот тип обеспечивает достаточную точность для реализованных алгоритмов и не требует параметризации всего публичного API.

Для параметров, которые легко перепутать с обычными числами, используется шаблон `StrongType`. На его основе определены типы `LR`, `Gain`, `Beta1`, `Beta2`, `Eps`. Обертка хранит значение и имеет явное создание из базового типа. Например, шаг обучения задается как `LR{0.001}`, а не как обычный `double`. Это делает вызовы конструкторов более читаемыми.

6.2 RandomGenerator

Класс `RandomGenerator` инкапсулирует `std::mt19937`. Он создается с `seed` по умолчанию или с явно заданным `seed`:

```
1 nns::RandomGenerator rng(nns::Seed{42});
```

Класс предоставляет методы `reseed`, `seed`, `init_matrix`, `shuffle` и `get_random_value`. Наиболее важным является метод `init_matrix`, который заполняет матрицу случайными значениями из нормального распределения.

Поддерживаются три схемы инициализации:

- `Normal`: стандартное отклонение равно `gain`;
- `XavierNormal`: стандартное отклонение равно $gain \sqrt{2/(fan_{in} + fan_{out})}$;
- `HeNormal`: стандартное отклонение равно $gain \sqrt{2/fan_{in}}$.

Инициализация `Xavier` предложена как способ стабилизировать дисперсию сигналов при прохождении через глубокую сеть [3]. Инициализация `He` адаптирована для сетей с `rectifier`-активациями, в частности с `ReLU` [5].

Метод `init_matrix` проверяет, что матрица непустая, а `gain` положителен. Проверка непустоты важна, поскольку формулы инициализации используют размеры матрицы, а заполнение пустой матрицы обычно свидетельствует об ошибке в конфигурации слоя.

7 Функции потерь

7.1 Общий контракт

Модуль функций потерь состоит из встроенных реализаций и `type-erased` обертки `AnyLossFunction`. Интерфейс функции потерь описан в разделе о стирании типов; здесь рассматриваются свойства встроенных реализаций. Проверка формы вынесена во вспомогательную функцию `validate_same_nonempty_shape`. Все встроенные функции потерь требуют, чтобы `y_hat` и `y` имели одинаковую форму и были непустыми. Функция потерь возвращает градиент той же формы, что и выход сети, чтобы его можно было передать в `NeuralNetwork::backward`.

7.2 Loss-функции для регрессии

Для регрессионных задач реализованы `MSELoss`, `MAELoss` и `HuberLoss`. `MSE` чувствительна к большим ошибкам, поскольку ошибка возводится в квадрат. `MAE` менее чувствительна к выбросам, но ее градиент имеет разрыв в нуле. `Huber loss` объединяет свойства `MSE` и `MAE`: при малой ошибке она квадратична, а при большой ошибке растет линейно. Поэтому `Huber loss` часто используют как устойчивую альтернативу `MSE`.

В реализации `HuberLoss` параметр `delta` является публичным полем со значением по умолчанию 1.0. Перед вычислением значения и градиента проверяется, что `delta > 0`. Это пример локальной валидации гиперпараметров: некорректное значение обнаруживается в функции, для которой оно имеет смысл.

7.3 Loss-функции для классификации

Для бинарной классификации `BCELoss` работает с вероятностями. Чтобы избежать вычисления $\log(0)$, предсказания ограничиваются снизу и сверху через `eps`. Значение `eps` должно лежать в интервале $(0, 0.5)$: при `eps <= 0` защита не работает, а при `eps >= 0.5` отсечение разрушает смысл вероятностей.

`BCEWithLogitsLoss` принимает не вероятности, а logits. Это предпочтительно, когда последним слоем модели является линейный слой без сигмоиды. Объединение сигмоиды и бинарной кросс-энтропии в одной формуле улучшает численную устойчивость и уменьшает риск переполнения.

Для многоклассовой классификации `CrossEntropyLoss` принимает logits и one-hot разметку. Softmax применяется внутри функции потерь, а градиент имеет простой вид:

$$\frac{\partial L}{\partial z} = \frac{p - y}{B},$$

где p — результат softmax, y — one-hot вектор класса, B — размер батча. Такое объединение softmax и кросс-энтропии соответствует типичному устройству современных библиотек глубокого обучения.

8 Оптимизаторы и scheduler-ы

8.1 Learning rate scheduler

Scheduler отвечает только за значение шага обучения на текущей итерации. В проекте реализованы `ConstantLR` и `TimeDecayLR`. `ConstantLR` возвращает постоянное значение, а `TimeDecayLR` вычисляет

$$\eta_t = \eta_0 \left(\frac{s_0}{s_0 + t} \right)^p.$$

Здесь η_0 — начальный шаг, t — номер итерации, s_0 и p — параметры убывания. В конструкторе можно задать любой из этих параметров, кроме t .

Оба scheduler-а хранят счетчик итераций. Оптимизатор вызывает `iter_step` после обработки каждого батча. Это означает, что расписание шага обучения изменяется по итерациям оптимизации, а не по эпохам.

8.2 SGDOptimizer

Стохастический градиентный спуск обновляет параметр по правилу

$$\theta_{t+1} = \theta_t - \eta_t \nabla_{\theta} L.$$

В реализации `SGDOptimizer` один и тот же шаблонный приватный метод `update_any` применяется к матрицам и векторам. Перед обновлением проверяется совпадение формы параметра и градиента. Состояние оптимизатора для SGD отсутствует, поэтому метод `update_weights` возвращает пустой `std::any`.

8.3 AdamOptimizer

Adam использует экспоненциальные скользящие средние первого и второго моментов градиента [6]. Для градиента g_t обновления имеют вид

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2. \end{aligned}$$

Затем применяется коррекция смещения:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t},$$

и параметр обновляется по правилу

$$\theta_{t+1} = \theta_t - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

В реализации состояние Adam хранится отдельно для каждого параметра в кэше оптимизатора. Для матрицы весов и вектора смещения линейного слоя создаются независимые кэши. При первом обновлении, когда кэш пуст, моменты инициализируются нулями той же формы, что и градиент.

Значения `beta1`, `beta2` и `eps` проверяются перед шагом Adam. Параметры `beta1` и `beta2` должны лежать в диапазоне $[0, 1]$, а `eps` должен быть положительным. Нарушение этих ограничений приводит к исключению `std::invalid_argument`.

9 Работа с данными

9.1 DataLoader

Класс `DataLoader` хранит матрицы признаков и целевых значений, размер батча, флаг перемешивания и вектор индексов. Конструктор принимает матрицы по значению и перемещает их во внутренние поля:

```
1 nns::DataLoader loader(  
2     std::move(X),  
3     std::move(Y),  
4     nns::BatchSize{64},  
5     nns::Shuffle{true});
```

При создании проверяется, что число объектов в `X` и `Y` совпадает, размер батча положителен, а набор данных не пуст. Число объектов определяется по числу столбцов. Метод `num_batches` возвращает только число полных батчей, то есть остаток в конце эпохи отбрасывается.

Метод `get_batch` создает новые матрицы батча и копирует в них столбцы исходных матриц в порядке, заданном вектором индексов. Если включено перемешивание, метод `reset_epoch(RandomGenerator& rng)` перемешивает индексы. Вызов `reset_epoch()` без генератора допустим только при `Shuffle{false}`; иначе библиотека не может выполнить воспроизводимое перемешивание и выбрасывает `std::logic_error`.

9.2 CSV-загрузка

Функция `load_csv` загружает числовой CSV-файл и разделяет колонки на признаки и целевые значения. Пользователь передает путь, множество индексов целевых колонок, разделитель и флаг наличия заголовка:

```
1 auto [X, Y] = nns::load_csv("data.csv", {0}, ',', true);
```

Если первая колонка является меткой класса, а остальные колонки являются признаками, вызов с `{0}` вернет матрицу признаков без первой колонки и матрицу целей из первой колонки. Данные в итоговых матрицах хранятся по принятому соглашению: объекты становятся столбцами.

Функция выполняет несколько проверок. Множество целевых колонок не должно быть пустым; файл должен открываться; файл должен содержать хотя бы одну строку данных; все строки должны иметь одинаковое число колонок; индексы целевых колонок должны попадать в диапазон; должна существовать хотя бы одна feature-колонка. Эти проверки особенно важны для CSV, поскольку ошибки формата данных иначе проявились бы позже как ошибки размерности матриц.

10 Trainer и цикл обучения

10.1 Роль Trainer

Класс `Trainer` связывает четыре объекта: нейронную сеть, оптимизатор, функцию потерь и загрузчик данных. Он хранит ссылки на эти объекты, поэтому они должны жить дольше самого тренера. Такой выбор предотвращает дорогие копирования сети и данных, а также делает владение объектами явным на стороне пользователя.

Конструктор имеет вид:

```
1 nns::Trainer trainer(net, opt, loss, loader);
```

Объект `Trainer` предоставляет методы `fit_epoch`, `fit`, а также `reset_optimizer_state`. Методы `fit_epoch` обучают одну эпоху, а методы `fit` обучают сразу все эпохи и возвращает вектор средних значений функции потерь по эпохам

10.2 Алгоритм одной эпохи

Одна эпоха обучения состоит из следующих шагов:

1. сбросить эпоху в `DataLoader` и при необходимости перемешать индексы;
2. для каждого полного батча получить матрицы `bX` и `bY`;
3. выполнить `net.forward` и получить предсказание и кэш слоев;

4. вычислить значение функции потерь;
5. вычислить градиент функции потерь по предсказанию;
6. выполнить `net.backward` и получить градиенты слоев;
7. выполнить `net.update`, передав градиенты, оптимизатор и кэш оптимизатора;
8. вызвать `opt.iter_step`;
9. добавить значение `loss` к сумме по эпохе.

После обработки всех батчей возвращается среднее значение функции потерь по числу батчей. Если `DataLoader` не содержит ни одного полного батча, тренер выбрасывает `std::logic_error`.

10.3 Состояние оптимизатора

Состояние оптимизатора хранится внутри `Trainer` в поле `std::any opt_cache_`. Для SGD состояние пустое, а для Adam содержит моменты для каждого параметра каждого слоя. Такое размещение состояния логично: состояние оптимизации относится не к самой модели как математической функции, а к конкретному процессу обучения.

Метод `reset_optimizer_state` очищает кэш. Это полезно, если одну и ту же сеть нужно продолжить обучать с заново инициализированным оптимизатором или если пользователь меняет оптимизатор между экспериментами.

11 Контракты API и обработка ошибок

Компоненты библиотеки проверяют входные параметры на границах публичного API. Это касается размерностей матриц, положительности гиперпараметров, согласованности числа батчей, наличия данных и доступности внешних файлов. Проверки выполняются рядом с тем кодом, который обладает достаточным контекстом для точного сообщения об ошибке. Например, слой проверяет размерности своего входа, функция потерь проверяет совпадение формы предсказания и целевой матрицы, а `DataLoader` проверяет размер батча и число объектов.

Такое распределение проверок сохраняет независимость компонентов. Линейный слой можно тестировать без тренера, функцию потерь — без сети, оптимизатор — на одной матрице или одном векторе. Полный интеграционный тест нужен только для проверки того, что компоненты корректно соединяются в цикле обучения.

11.1 Стратегия обработки ошибок

В библиотеке используются исключения `std::invalid_argument`, `std::logic_error`, `std::out_of_range` и `std::runtime_error`. Их смысл различается:

- `std::invalid_argument` означает некорректное значение аргумента: неправильная форма матрицы, неположительный `learning rate`, неверный параметр функции потерь;
- `std::logic_error` означает неправильный порядок действий: например, вызов `fit_epoch()` без генератора при включенном перемешивании;
- `std::out_of_range` используется для обращения к несуществующему батчу;
- `std::runtime_error` используется для ошибок внешней среды, например невозможности открыть CSV-файл.

Такое разделение делает сообщения об ошибках полезнее для пользователя. Например, ошибка размера матрицы обычно исправляется в коде конфигурации модели, а ошибка открытия CSV-файла может быть связана с путями, рабочей директорией или отсутствием данных.

12 Размерности, сложность и численная устойчивость

12.1 Соглашения о размерностях

Большая часть ошибок при ручной реализации нейронных сетей связана с перепутанными размерностями. В проекте принято соглашение, что объекты выборки хранятся в столбцах. Поэтому линейный слой с входной размерностью d_{in} и выходной размерностью d_{out} работает со следующими объектами:

Объект	Смысл	Размерность
X	вход батча	$d_{in} \times B$
A	матрица весов	$d_{out} \times d_{in}$
b	вектор смещения	$d_{out} \times 1$
Y	выход батча	$d_{out} \times B$
dY	градиент по выходу	$d_{out} \times B$
dA	градиент по весам	$d_{out} \times d_{in}$
db	градиент по смещению	$d_{out} \times 1$
dX	градиент по входу	$d_{in} \times B$

Именно эти размерности проверяются в коде. Например, `LinearLayer::predict` требует, чтобы число строк входной матрицы совпадало с числом столбцов матрицы весов. `LinearLayer::backward` проверяет, что число строк dY совпадает с выходной размерностью, а число столбцов совпадает с размером батча, сохраненным в кэше.

12.2 Асимптотическая сложность

Основная вычислительная стоимость полносвязной сети приходится на матричные умножения. Для одного линейного слоя прямой проход требует умножения матрицы $d_{out} \times d_{in}$ на матрицу $d_{in} \times B$, что имеет сложность

$$O(d_{out}d_{in}B).$$

Прибавление смещения имеет сложность $O(d_{out}B)$ и обычно существенно дешевле умножения.

Обратный проход линейного слоя содержит два матричных умножения: вычисление $dA = dYX^T$ и вычисление $dX = A^TdY$. Каждое из них также имеет порядок $O(d_{out}d_{in}B)$. Следовательно, обучение одного линейного слоя на одном батче имеет ту же асимптотику, что и несколько матричных умножений с теми же размерами. Для сети из m линейных слоев суммарная стоимость одной итерации равна сумме стоимостей по слоям.

Слой активации имеет сложность $O(n)$, где n — число элементов входной матрицы. Он дешевле линейного слоя, если размерности достаточно велики. Функции потерь также обычно имеют линейную сложность по числу элементов выхода. Исключением не является и `softmax`: он требует прохода по каждому столбцу для поиска максимума, вычисления экспонент и нормировки, что дает $O(kB)$ для k классов и размера батча B .

12.3 Память и кэши

Логика владения кэшами описана в архитектурной части отчета. С точки зрения памяти важно, что во время прямого прохода сохраняются данные, необходимые для последующего вычисления локальных градиентов. Поэтому память кэша прямого прохода имеет тот же порядок, что и сумма размеров входов всех слоев на данном батче.

Состояние Adam также требует дополнительной памяти. Для каждого параметра хранятся две величины той же формы: первый и второй моменты. Для линейного слоя это две матрицы размера $d_{out} \times d_{in}$ и два вектора размера d_{out} . Поэтому Adam примерно утраивает память, связанную с параметрами: сами параметры плюс два набора моментов. SGD такого состояния не имеет.

12.4 Численная устойчивость

В нескольких местах реализации явно учитывается численная устойчивость. В `CrossEntropyLoss::softmax` из каждого столбца `logits` вычитается максимум. Без этого при больших положительных `logits` экспонента может переполниться. Так как `softmax` инвариантен к прибавлению одной и той же константы ко всем компонентам вектора, такое преобразование не меняет математический результат.

В `BCELoss` вероятности ограничиваются через `eps`. Это защищает от логарифма нуля и деления на ноль в градиенте. В `BCEWithLogitsLoss` используется стабильная формула через $\max(z, 0)$ и $\log(1 + e^{-|z|})$, которая избегает прямого вычисления $\log(1 + e^z)$ при большом z .

В Adam параметр ε добавляется к знаменателю при делении на корень из второго момента. Это предотвращает деление на ноль и стабилизирует обновление при очень малых градиентах.

13 Пример пользовательского сценария

13.1 Минимальный пример обучения

Ниже приведен сокращенный пример обучения сети на небольшом наборе данных. Он показывает полный путь от создания данных до запуска Trainer.

```
1 #include <iostream>
2 #include <nns/NNS.hpp>
3
4 int main() {
5     nns::RandomGenerator rng(nns::Seed{42});
6
7     nns::Matrix X(2, 4);
8     X << 0.0, 0.0, 1.0, 1.0,
9         0.0, 1.0, 0.0, 1.0;
10
11     nns::Matrix Y = nns::Matrix::Zero(2, 4);
12     Y(0, 0) = 1.0;
13     Y(1, 1) = 1.0;
14     Y(1, 2) = 1.0;
15     Y(0, 3) = 1.0;
16
17     nns::DataLoader loader(std::move(X), std::move(Y),
18                           nns::BatchSize{4}, nns::Shuffle{true});
19
20     nns::NeuralNetwork net(
21         nns::LinearLayer(
22             nns::In{2},
23             nns::Out{8},
24             rng,
25             nns::Distribution::HeNormal),
26         nns::ReLU(),
27         nns::LinearLayer(
28             nns::In{8},
29             nns::Out{2},
30             rng,
31             nns::Distribution::XavierNormal));
32
33     nns::AnyOptimizer opt =
34         nns::make_AnyOptimizer(
35             nns::AdamOptimizer(
36                 nns::ConstantLR(nns::LR{0.01})));
37     nns::AnyLossFunction loss =
38         nns::make_AnyLossFunction<nns::CrossEntropyLoss>();
39
40     nns::Trainer trainer(net, opt, loss, loader);
41     std::vector<nns::Scalar> history = trainer.fit(200, rng);
42
43     std::cout << "last loss = " << history.back() << '\n';
44 }
```

В этом примере используется CrossEntropyLoss, поэтому последний слой возвращает logits, а softmax применяется внутри функции потерь. Для инференса вероятности можно получить вызовом CrossEntropyLoss::softmax от результата net.predict.

14 Экспериментальное обучение на MNIST

14.1 Конфигурация эксперимента

В файле main.cpp реализован эксперимент на наборе MNIST. MNIST содержит изображения рукописных цифр размера 28×28 и разметку по десяти классам [8]. В репозитории эксперимент рассчитан на CSV-представление данных: первая колонка содержит метку класса, остальные 784 колонки содержат значения пикселей.

Обработка данных выполняется следующим образом. Функция load_csv отделяет колонку метки от признаков. Значения пикселей нормируются делением на 255, то есть переводятся из диапазона $[0, 255]$ в диапазон $[0, 1]$. Метки классов преобразуются в one-hot представление размерности $10 \times N$. После этого обучающая выборка передается в DataLoader с размером батча 100 и включенным перемешиванием.

Эксперимент MNIST

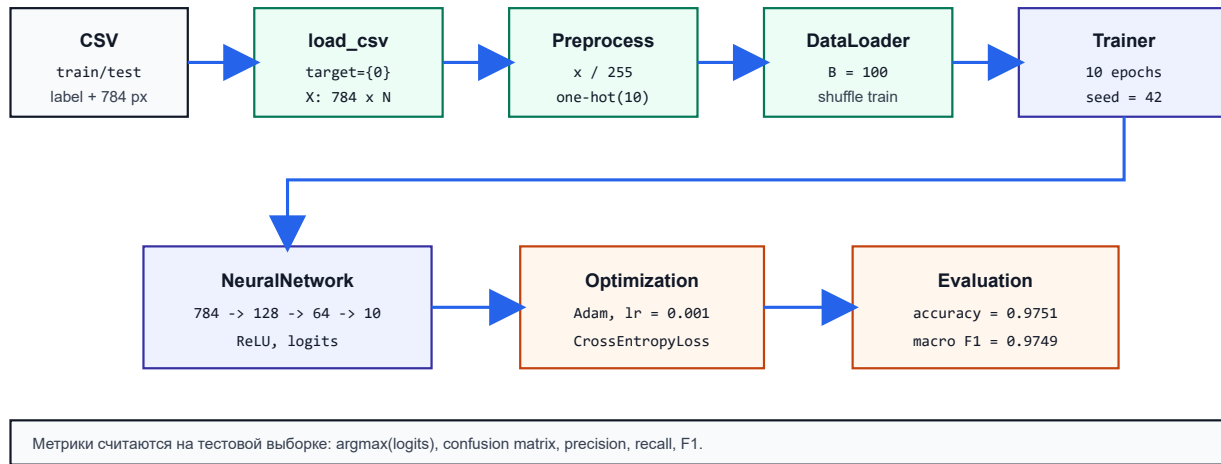


Рис. 4: Поток данных в эксперименте MNIST

Схема эксперимента показана на рисунке 4. Архитектура сети:

$784 \rightarrow 128 \rightarrow 64 \rightarrow 10$.

Между линейными слоями используются ReLU. Последний слой возвращает logits. В качестве функции потерь используется CrossEntropyLoss, которая применяет softmax внутри вычисления loss и gradient. Оптимизатор — Adam с постоянным learning rate 0.001. Число эпох равно 10, seed генератора равен 42.

Фрагмент кода, задающий конфигурацию обучения, приведен ниже.

```
1 RandomGenerator rng(Seed{42});
2
3 constexpr size_t batch_size = 100;
4 constexpr size_t epochs = 10;
5
6 DataLoader train_loader(
7     std::move(train_X),
8     std::move(train_Y),
9     BatchSize{batch_size},
10    Shuffle{true});
11
12 NeuralNetwork net(
13     LinearLayer(
14         In{784},
15         Out{128},
16         rng,
17         Distribution::Normal,
18         Gain{0.01}),
19     ReLU(),
20     LinearLayer(
21         In{128},
22         Out{64},
23         rng,
24         Distribution::Normal,
25         Gain{0.01}),
26     ReLU(),
27     LinearLayer(
28         In{64},
29         Out{10},
30         rng,
31         Distribution::Normal,
32         Gain{0.01}));
```

```

33
34 AnyOptimizer opt = make_AnyOptimizer(
35     AdamOptimizer(ConstantLR(LR{0.001})));
36
37 AnyLossFunction loss_func = make_AnyLossFunction<CrossEntropyLoss>();
38 Trainer trainer(net, opt, loss_func, train_loader);

```

14.2 Методика расчета метрик

Для оценки качества используется тестовая часть MNIST. Предсказанный класс определяется как индекс максимального logit:

$$\hat{c} = \arg \max_i z_i.$$

На основе истинного класса c и предсказанного класса $\hat{c}$ строится матрица ошибок C , где C_{ij} равно числу объектов истинного класса i , отнесенных моделью к классу j .

Ассигу вычисляется как доля правильно классифицированных объектов:

$$accuracy = \frac{\sum_i C_{ii}}{\sum_{i,j} C_{ij}}.$$

Для каждого класса дополнительно вычисляются precision, recall и F1:

$$precision_i = \frac{TP_i}{TP_i + FP_i}, \quad recall_i = \frac{TP_i}{TP_i + FN_i},$$

$$F1_i = \frac{2 \cdot precision_i \cdot recall_i}{precision_i + recall_i}.$$

Магсо-метрики являются средним арифметическим по классам, weighted-метрики являются средним с весами, равными числу объектов соответствующего класса.

Код расчета матрицы ошибок и ассигу использует тот же predict, который применяется в пользовательском режиме инференса:

```

1 Matrix logits = net.predict(batch_X);
2 for (Index col = 0; col < cols; ++col) {
3     Index pred_class = 0;
4     Index true_class = 0;
5     logits.col(col).maxCoeff(&pred_class);
6     batch_Y.col(col).maxCoeff(&true_class);
7     confusion[true_class][pred_class]++;
8     if (pred_class == true_class) {
9         ++correct;
10    }
11 }

```

14.3 Результаты обучения

Эксперимент запускался в конфигурации release. В debug-конфигурации тот же код выполняется существенно медленнее из-за отсутствия оптимизаций компилятора. История функции потерь на обучающей выборке и ассигу на тестовой выборке:

Эпоха	Train loss	Test accuracy
1	0.534551	0.9197
2	0.235347	0.9438
3	0.164380	0.9538
4	0.125233	0.9631
5	0.102050	0.9653
6	0.082142	0.9717
7	0.067856	0.9734
8	0.057028	0.9723
9	0.047776	0.9754
10	0.039908	0.9751

Итоговые метрики на тестовой выборке после 10 эпох:

Метрика	Значение
Test cross-entropy	0.084976
Accuracy	0.975100
Macro precision	0.975139
Macro recall	0.974866
Macro F1	0.974938
Weighted precision	0.975243
Weighted recall	0.975100
Weighted F1	0.975108

Матрица ошибок для тестовой выборки приведена ниже. Строки соответствуют истинному классу, столбцы — предсказанному классу.

	0	1	2	3	4	5	6	7	8	9
0	967	0	2	2	3	0	2	1	2	1
1	0	1122	1	1	0	0	2	1	7	1
2	3	4	997	8	5	1	1	7	6	0
3	0	0	1	996	2	3	0	4	1	3
4	1	0	0	0	962	0	4	4	1	10
5	2	0	0	13	2	867	4	1	2	1
6	5	2	1	0	15	8	926	0	1	0
7	1	5	7	4	1	0	0	1004	0	6
8	3	0	3	9	5	3	4	5	936	6
9	1	2	0	7	13	2	0	8	2	974

Полученные значения показывают, что реализованный обучающий цикл корректно снижает функцию потерь и формирует пригодную для классификации модель. Наиболее частые ошибки связаны с визуально близкими цифрами, например с переходами между классами 4 и 9, 5 и 3, 7 и 2.

15 Сборка проекта

15.1 Сборка через CMake

Проект собирается через CMake. В корневом `CMakeLists.txt` объявляется проект `NNS`, подключаются зависимости через `FetchContent`, создается `INTERFACE`-цель `nns` и псевдоним `NNS::nns`. Для библиотеки задается требование `cxx_std_23`.

В проекте предусмотрены опции:

- `NNS_BUILD_EXAMPLE` — собирать пример из `main.cpp`;
- `NNS_BUILD_TESTING` — собирать тесты;
- `NNS_ENABLE_FORMAT_TARGETS` — включить цели форматирования.

Файл `CMakePresets.json` определяет пресеты `debug` и `release`. Оба используют `Ninja`, включают тесты и экспортируют `compile_commands.json`. Для удобства добавлен `Justfile` с командами `configure`, `build`, `test`, `format`, `format-check` и `check`.

15.2 Сборка через just

Так же поддерживается сборка через `just`. Для сборки в `debug` режиме используются следующие команды:

```
1 just configure
2 just build
3 just test
```

Для прогона по тестам и применение `clang-format` используется:

```
1 just check
```

Для `release` сборки к командам из `debug` режима добавляется флаг `release`:

```
1 just configure release
2 just build release
3 just test release
```

16 Тестирование

16.1 Организация тестового набора

Тесты разделены по функциональным областям:

- `activation_tests.cpp` проверяет встроенные и пользовательские активации;
- `core_tests.cpp` проверяет базовые типы;
- `layer_network_trainer_tests.cpp` проверяет слои, сеть и тренер;
- `loss_tests.cpp` проверяет функции потерь;
- `optimizer_tests.cpp` проверяет scheduler-ы и оптимизаторы;
- `utils_tests.cpp` проверяет генератор случайных чисел, `DataLoader` и CSV-загрузку.

16.2 Проверяемые сценарии

Тестовый набор покрывает три группы сценариев. Первая группа — проверка математических значений на малых входах. Например, для `MSELoss`, `MAELoss`, `HuberLoss`, `BCELoss` и `CrossEntropyLoss` проверяются значения функции потерь и отдельные элементы градиента. Для `ReLU`, `Sigmoid` и `Tanh` проверяются значения функции и производной.

Вторая группа — проверка контрактов размерностей. Тесты вызывают слои, сеть, функции потерь и оптимизаторы с некорректными формами матриц и ожидают исключения `std::invalid_argument`. Например, `LinearLayer::predict` должен отклонять вход с неправильным числом строк, `LinearLayer::backward` должен отклонять градиент неправильной формы, а `NeuralNetwork::backward` должен отклонять кэш с числом элементов, не совпадающим с числом слоев.

Третья группа — интеграционные сценарии. В них проверяется, что `NeuralNetwork` поддерживает полный цикл `predict -> forward -> backward -> update`, а `Trainer` выполняет эпоху обучения, возвращает историю `loss`, поддерживает перемешивание через `RandomGenerator` и отклоняет `DataLoader` , не содержащий полного батча.

16.3 Запуск тестов

Тесты запускаются через `CTest`:

```
1 cmake --preset debug
2 cmake --build --preset debug
3 ctest --preset debug
```

Также предусмотрена команда `just check`, которая собирает проект, запускает тесты и проверяет форматирование:

```
1 just check
```

На текущей версии проекта команда `ctest --preset debug` выполняет 29 тестов; все 29 тестов завершаются успешно.

16.4 Статические соглашения о стиле

Для форматирования используется `clang-format`. Цели `format` и `format-check` создаются CMake при наличии исполняемого файла `clang-format`. В репозитории также присутствует конфигурация `.clang-tidy`, ориентированная на группы проверок `google-*`, `cpcoreguidelines-*` и `readability-*`. Часть проверок, связанных с `magic numbers`, отключена, поскольку в тестах и численных формулах небольшие константы часто являются частью математического определения.

17 Текущие ограничения

17.1 Ограничения модели

Текущая версия библиотеки поддерживает только последовательные полносвязные сети. В ней отсутствуют сверточные слои, рекуррентные слои, residual-соединения, ветвления вычислительного графа и автоматическое дифференцирование произвольных операций. Это осознанное ограничение: проект фокусируется на понятной реализации базового обучения многослойного перцептрона.

Функции активации являются только скалярными. Поэтому операции, зависящие от всего вектора, например softmax как отдельный слой, не входят в интерфейс `ActivationLayer`. В текущей реализации softmax расположен внутри `CrossEntropyLoss`. Для задач, где softmax нужен как самостоятельная операция инференса, пользователь может вызвать `CrossEntropyLoss::softmax` явно.

17.2 Ограничения производительности

Библиотека ориентирована на CPU и использует Eigen. В ней нет поддержки GPU, смешанной точности, распараллеливания на уровне батчей или специализированных BLAS-бэкендов. Кроме того, кэши и батчи создают дополнительные копии матриц. Для больших моделей потребуется оптимизация владения памятью и уменьшение числа временных объектов.

18 Заключение

В рамках проекта разработана библиотека NNS для построения и обучения полносвязных нейронных сетей на C++23. Реализованы генератор случайных чисел, линейные слои, слои активации, последовательная нейронная сеть, функции потерь, scheduler-ы шага обучения, оптимизаторы SGD и Adam, загрузчик данных, CSV-парсер и обучающий цикл.

Ключевым архитектурным решением стало использование `type erasure` через библиотеку `rgxh`. Благодаря этому пользователь может расширять библиотеку собственными функциями активации, функциями потерь, оптимизаторами и scheduler-ами без наследования от библиотечных базовых классов. Вторым важным решением стало явное возвращение кэша прямого прохода и состояния оптимизатора через `std::any`; это делает поток данных между forward, backward и update прозрачным и локализует состояние обучения в `Trainer`.

Реализация покрыта модульными тестами, которые проверяют корректные сценарии и ошибки нарушения контрактов. Проект собирается через CMake, поддерживает пресеты debug/release, форматирование через clang-format и подключение как `NNS::nns`. В результате реализован полный стек обучения полносвязной нейронной сети: от загрузки и батчинга данных до вычисления градиентов и обновления параметров оптимизатором.

Список литературы

- [1] Catch2. Catch2: A modern c++ test framework. <https://github.com/catchorg/Catch2>, 2026. URL: <https://github.com/catchorg/Catch2>.
- [2] Eigen Project. Eigen: A c++ template library for linear algebra. <https://libeigen.gitlab.io/>, 2026. URL: <https://libeigen.gitlab.io/>.
- [3] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, volume 9 of *Proceedings of Machine Learning Research*, pages 249–256. PMLR, 2010. URL: <https://proceedings.mlr.press/v9/glorot10a.html>.
- [4] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL: <https://www.deeplearningbook.org/>.
- [5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1026–1034, 2015. URL: https://www.cv-foundation.org/openaccess/content_iccv_2015/html/He_Delving_Deep_into_ICCV_2015_paper.html.
- [6] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. arXiv:1412.6980, <https://arxiv.org/abs/1412.6980>, 2014. URL: <https://arxiv.org/abs/1412.6980>.

- [7] Kitware. *CMake FetchContent Module Documentation*, 2026. URL: <https://cmake.org/cmake/help/latest/module/FetchContent.html>.
- [8] Yann LeCun, Corinna Cortes, and Christopher J. C. Burges. The mnist database of handwritten digits. <https://yann.lecun.org/exdb/mnist/>, 1998. URL: <https://yann.lecun.org/exdb/mnist/>.
- [9] Microsoft. Proxy 4: Pointer-semantics-based polymorphism for c++. <https://microsoft.github.io/proxy/>, 2026. URL: <https://microsoft.github.io/proxy/>.
- [10] Michael A. Nielsen. *Neural Networks and Deep Learning*. Determination Press, 2015. URL: <http://neuralnetworksanddeeplearning.com/>.